

# Instruction set architecture

---

An **instruction set architecture (ISA)** is an abstract model that defines the programmable interface of the CPU of a computer, defining how software interacts with hardware.<sup>[1]</sup> A device (i.e. CPU) that interprets instructions described by an ISA is an implementation of that ISA. Generally, the same ISA is used for a family of related CPU devices.

In general, an ISA defines the instructions, data types, registers, and the programming interface for managing main memory such as addressing modes, virtual memory, and memory consistency mechanisms. The ISA also includes the input/output model of the programmable interface.

An ISA specifies the behavior implied by machine code running on an implementation of that ISA in a fashion that does not depend on the characteristics of that implementation, providing binary compatibility between implementations. This enables multiple implementations of an ISA that differ in characteristics such as performance, physical size, and monetary cost (among other things), but that are capable of running the same machine code, so that a lower-performance, lower-cost machine can be replaced with a higher-cost, higher-performance machine without having to replace software. It also enables the evolution of the microarchitectures of the implementations of that ISA, so that a newer, higher-performance implementation of an ISA can run software that runs on previous generations of implementations.

If an operating system maintains a standard and compatible application binary interface (ABI) for a particular ISA, machine code will run on future implementations of that ISA and operating system. However, if an ISA supports running multiple operating systems, it does not guarantee that machine code for one operating system will run on another operating system, unless the first operating system supports running machine code built for the other operating system.

An ISA can be extended by adding instructions or other capabilities, or adding support for larger addresses and data values; an implementation of the extended ISA will still be able to execute machine code for versions of the ISA without those extensions. Machine code using those extensions will only run on implementations that support those extensions.

The binary compatibility that they provide makes ISAs one of the most fundamental abstractions in computing.

## Overview

---

An instruction set architecture is distinguished from a microarchitecture, which is the set of processor design techniques used, in a particular processor, to implement the instruction set. Processors with different microarchitectures can share a common instruction set. For example, the Intel Pentium and the AMD Athlon implement nearly identical versions of the x86 instruction set, but they have radically different internal designs.

The concept of an *architecture*, distinct from the design of a specific machine, was developed by Fred Brooks at IBM during the design phase of System/360.

Prior to NPL [System/360], the company's computer designers had been free to honor cost objectives not only by selecting technologies but also by fashioning functional and architectural refinements. The SPREAD compatibility objective, in contrast, postulated a single architecture for a series of five processors spanning a wide range of cost and performance. None of the five engineering design teams could count on being able to bring about adjustments in architectural specifications as a way of easing difficulties in achieving cost and performance objectives.<sup>[2]:p.137</sup>

Some virtual machines that support bytecode as their ISA such as Smalltalk, the Java virtual machine, and Microsoft's Common Language Runtime, implement this by translating the bytecode for commonly used code paths into native machine code. In addition, these virtual machines execute less frequently used code paths by interpretation (see: Just-in-time compilation). Transmeta implemented the x86 instruction set atop very long instruction word (VLIW) processors in this fashion.

## Classification of ISAs

---

An ISA may be classified in a number of different ways. A common classification is by architectural *complexity*. A complex instruction set computer (CISC) has many specialized instructions, some of which may only be rarely used in practical programs. A reduced instruction set computer (RISC) simplifies the processor by efficiently implementing only the instructions that are frequently used in programs, while the less common operations are implemented as subroutines, having their resulting additional processor execution time offset by infrequent use.<sup>[3]</sup>

Other types include VLIW architectures, and the closely related long instruction word (LIW) and explicitly parallel instruction computing (EPIC) architectures. These architectures seek to exploit instruction-level parallelism with less hardware than RISC and CISC by making the compiler responsible for instruction issue and scheduling.<sup>[4]</sup>

Architectures with even less complexity have been studied, such as the minimal instruction set computer (MISC) and one-instruction set computer (OISC). These are theoretically important types, but have not been commercialized.<sup>[5][6]</sup>

## Instructions

---

Machine language is built up from discrete *statements* or *instructions*. On the processing architecture, a given instruction may specify:

- opcode (the instruction to be performed) e.g. add, copy, test
- any explicit operands:

- registers
  - literal/constant values
  - addressing modes used to access memory

More complex operations are built up by combining these simple instructions, which are executed sequentially, or as otherwise directed by control flow instructions.

## Instruction types

Examples of operations common to many instruction sets include:

### Data handling and memory operations

- Set a register or memory to a fixed constant value.
- *Copy* data from a one place to another. This operation is often called load or store. Although the machine instruction is often called *move*, the term is misleading because the source remains unchanged. These operations are used to store the contents of a register, the contents of another memory location or the result of a computation, or to retrieve stored data to perform a computation later.
- *Read* or *write* data from hardware devices.

### Arithmetic and logic operations

- *Add, subtract, multiply, or divide* the values of two registers, placing the result in a register, possibly setting one or more condition codes in a status register.<sup>[7]</sup>
  - *increment, decrement* in some ISAs, saving operand fetch in trivial cases.
- Perform bitwise operations, e.g., taking the conjunction and disjunction of corresponding bits in a pair of registers, taking the negation of each bit in a register.
- *Compare* two values in registers (for example, to see if one is less, or if they are equal).
- *Floating-point instructions* for arithmetic on floating-point numbers.<sup>[7]</sup>

### Control flow operations

- Branch to another location in the program and execute instructions there.
- Conditionally branch to another location if a certain condition holds.
- Indirectly branch to another location.
- *Skip* one or more instructions, depending on conditions (a conditional branch a fixed number of instructions forward)
- *Trap* Explicitly cause a software interrupt, either conditionally or unconditionally.
- *Call* another block of code, while saving the location of the next instruction as a point to return to.
- Return from a previous call by retrieving the saved location.

### Coprocessor instructions

- Load/store data to and from a coprocessor or exchanging with CPU registers.
- Perform coprocessor operations.

Some examples of coprocessor instructions include those for the IBM 3090 Vector facility and the Intel 8087.

## Elaborate instructions

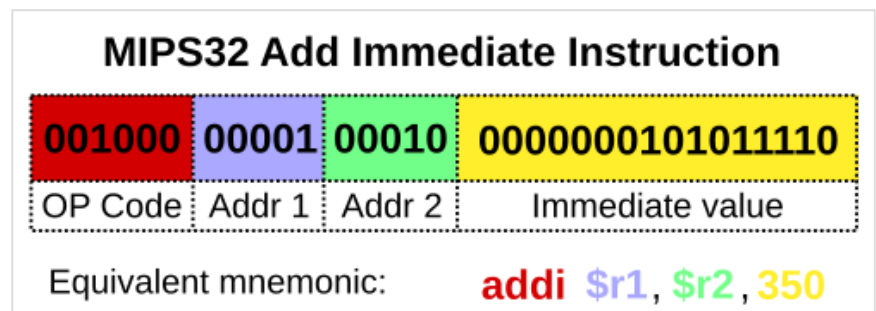
Processors may include elaborate instructions in their instruction set. A single elaborate instruction does something that may require many instructions or a subroutine on other computers. Such instructions are typified by instructions that take multiple steps, control multiple functional units, or otherwise appear on a larger scale than the bulk of simple instructions implemented by the given processor. Some examples of elaborate instructions include:

- transferring multiple registers to or from memory (especially the stack) at once
- moving large blocks of memory (e.g. string copy or DMA transfer)
- searching strings for matches or differences
- arithmetic on BCD strings
- complicated integer and floating-point arithmetic (e.g. square root, or transcendental functions such as logarithm, sine, cosine, etc.)
- SIMD instructions, a single instruction performing an operation on many homogeneous values in parallel, possibly in dedicated SIMD registers

Elaborate instructions are more common in CISC instruction sets than in RISC instruction sets, but RISC instruction sets may include them as well. RISC instruction sets generally do not include ALU operations with memory operands, or instructions to move large blocks of memory, but most RISC instruction sets include SIMD or vector instructions that perform the same arithmetic operation on multiple pieces of data at the same time. SIMD instructions have the ability of manipulating large vectors and matrices in minimal time. SIMD instructions allow easy parallelization of algorithms commonly involved in sound, image, and video processing. Various SIMD implementations have been brought to market under trade names such as MMX, 3DNow!, and AltiVec.

## Instruction encoding

On traditional architectures, an instruction includes an opcode that specifies the operation to perform, such as *add contents of memory to register*—and zero or more operand specifiers, which may specify registers, memory locations, or literal data. The operand specifiers may have addressing modes determining their meaning or may be in fixed fields. In very long instruction word (VLIW) architectures, which include many microcode architectures, multiple simultaneous opcodes and operands are specified in a single instruction.



One instruction may have several fields, which identify the logical operation, and may also include source and destination addresses and constant values. This is the MIPS "Add Immediate" instruction, which allows selection of source and destination registers and inclusion of a small constant.

Some exotic instruction sets do not have an opcode field, such as transport triggered architectures (TTA), only operand(s).

Most stack machines have "0-operand" instruction sets in which arithmetic and logical operations lack any operand specifier fields; only instructions that push operands onto the evaluation stack or that pop operands from the stack into variables have operand specifiers. The instruction set carries out most ALU actions with postfix (reverse Polish notation) operations that work only on the expression stack, not on data registers or arbitrary main memory cells. This can be very convenient for compiling high-level languages, because most arithmetic expressions can be easily translated into postfix notation.<sup>[8]</sup>

## Conditional instructions

Conditional instructions often have a predicate field—a few bits that encode the specific condition to cause an operation to be performed rather than not performed. For example, a conditional branch instruction will transfer control if the condition is true, so that execution proceeds to a different part of the program, and not transfer control if the condition is false, so that execution continues sequentially. Some instruction sets also have conditional moves, so that the move will be executed, and the data stored in the target location, if the condition is true, and not executed, and the target location not modified, if the condition is false. Similarly, IBM z/Architecture has a conditional store instruction. A few instruction sets include a predicate field in every instruction. Having predicates on instructions is called predication, and can *include* conditional-branches, such as `b f` on the SuperH.<sup>[9]</sup>

## Loop control instructions

Many architectures have conditional branch instructions designed for controlling loops. Typically, they update and test a register. Examples include

### IBM 7070

- Branch decremented index word (BDX)
- Branch incremented index word (BIX)

### IBM 7090

- Transfer on index (TIX)
- Transfer on index high (TXH)
- Transfer on index low or equal (TXL)

### IBM System/360

- Branch on count (BCT, BCTR)
- Branch on index high (BXH)
- Branch on index low or equal (BXLE)

## Number of operands

Instruction sets may be categorized by the maximum number of operands *explicitly* specified in instructions.

(In the examples that follow, *a*, *b*, and *c* are (direct or calculated) addresses referring to memory cells, while *reg1* and so on refer to machine registers.)

$$C = A + B$$

- 0-operand (*zero-address machines*), so called stack machines: All arithmetic operations take place using the top one or two positions on the stack:<sup>[10]</sup> `push a, push b, add, pop c`.

- $C = A+B$  needs *four instructions*.<sup>[11]</sup> For stack machines, the terms "0-operand" and "zero-address" apply to arithmetic instructions, but not to all instructions, as 1-operand push and pop instructions are used to access memory.
- 1-operand (*one-address machines*), so called accumulator machines, include early computers and many small microcontrollers: most instructions specify a single right operand (that is, constant, a register, or a memory location), with the implicit accumulator as the left operand (and the destination if there is one): load *a*, add *b*, store *c*.
  - $C = A+B$  needs *three instructions*.<sup>[11]</sup>
- 2-operand — many CISC and RISC machines fall under this category:
  - CISC — move *A* to *C*; then add *B* to *C*.
    - $C = A+B$  needs *two instructions*. This effectively 'stores' the result without an explicit *store* instruction.
  - CISC — Often machines are limited to one memory operand (<https://web.archive.org/web/20131105155703/http://cs.smith.edu/~thiebaut/ArtOfAssembly/CH04/CH04-3.html#HEADING3-79>) per instruction: load *a*, reg1; add *b*, reg1; store reg1, *c*; This requires a load/store pair for any memory movement regardless of whether the add result is an augmentation stored to a different place, as in  $C = A+B$ , or the same memory location:  $A = A+B$ .
    - $C = A+B$  needs *three instructions*.
  - RISC — Requiring explicit memory loads, the instructions would be: load *a*, reg1; load *b*, reg2; add reg1, reg2; store reg2, *c*.
    - $C = A+B$  needs *four instructions*.
- 3-operand, allowing better reuse of data:<sup>[12]</sup>
  - CISC — It becomes either a single instruction: add *a*, *b*, *c*
    - $C = A+B$  needs *one instruction*.
  - CISC — Or, on machines limited to two memory operands per instruction, move *a*, reg1; add reg1, *b*, *c*;
    - $C = A+B$  needs *two instructions*.
  - RISC — arithmetic instructions use registers only, so explicit 2-operand load/store instructions are needed: load *a*, reg1; load *b*, reg2; add reg1+reg2->reg3; store reg3, *c*;
    - $C = A+B$  needs *four instructions*.
    - Unlike 2-operand or 1-operand, this leaves all three values *a*, *b*, and *c* in registers available for further reuse.<sup>[12]</sup>
- more operands—some CISC machines permit a variety of addressing modes that allow more than 3 operands (registers or memory accesses), such as the VAX "POLY" polynomial evaluation instruction.

Due to the large number of bits needed to encode the three registers of a 3-operand instruction, RISC architectures that have 16-bit instructions are invariably 2-operand designs, such as the Atmel AVR, TI MSP430, and some versions of ARM Thumb. RISC architectures that have 32-bit instructions are usually

3-operand designs, such as the ARM, AVR32, MIPS, Power ISA, and SPARC architectures. However even 3-operand RISC architectures will, at considerable cost, have Fused multiply-and-add 4-operand instructions out of necessity, due to the increased accuracy provided. Modern examples include Power ISA and RISC-V.

Each instruction specifies some number of operands (registers, memory locations, or immediate values) *explicitly*. Some instructions give one or both operands implicitly, such as by being stored on top of the stack or in an implicit register. If some of the operands are given implicitly, fewer operands need be specified in the instruction. When a "destination operand" explicitly specifies the destination, an additional operand must be supplied. Consequently, the number of operands encoded in an instruction may differ from the mathematically necessary number of arguments for a logical or arithmetic operation (the arity). Operands are either encoded in the "opcode" representation of the instruction, or else are given as values or addresses following the opcode.

## Register pressure

*Register pressure* measures the availability of free registers at any point in time during the program execution. Register pressure is high when a large number of the available registers are in use. Thus, the higher the register pressure, the more often the register contents must be spilled into cache or memory which, given their slower speed, exacts a heavy price. Increasing the number of registers in an architecture decreases register pressure but increases the cost.<sup>[13]</sup>

While embedded instruction sets such as Thumb suffer from extremely high register pressure because they have small register sets, general-purpose RISC ISAs like MIPS and Alpha enjoy low register pressure. CISC ISAs like x86-64 offer low register pressure despite having smaller register sets. This is due to the many addressing modes and optimizations (such as sub-register addressing, memory operands in ALU instructions, absolute addressing, PC-relative addressing, and register-to-register spills) that CISC ISAs offer.<sup>[14]</sup>

## Instruction length

The size or length of an instruction varies widely, from as little as four bits in some microcontrollers to many hundreds of bits in some VLIW systems. Processors used in personal computers, mainframes, and supercomputers have minimum instruction sizes between 8 and 64 bits. The longest possible instruction on x86 is 15 bytes (120 bits).<sup>[15]</sup> Within an instruction set, different instructions may have different lengths. In some architectures, notably most reduced instruction set computers (RISC), instructions are a fixed length, typically corresponding with that architecture's word size. In other architectures, instructions have variable length, typically integral multiples of a byte or a halfword. Some, such as the ARM with *Thumb-extension* have *mixed* variable encoding, that is two fixed, usually 32-bit and 16-bit encodings, where instructions cannot be mixed freely but must be switched between on a branch (or exception boundary in ARMv8).

Fixed-length instructions are less complicated to handle than variable-length instructions for several reasons (not having to check whether an instruction straddles a cache line or virtual memory page boundary,<sup>[12]</sup> for instance), and are therefore somewhat easier to optimize for speed.

## Code density

In early 1960s computers, main memory was expensive and very limited, even on mainframes. Minimizing the size of a program to make sure it would fit in the limited memory was often central. Thus the size of the instructions needed to perform a particular task, the *code density*, was an important characteristic of any instruction set. It remained important on the initially-tiny memories of minicomputers and then microprocessors. Density remains important today, for smartphone applications, applications downloaded into browsers over slow Internet connections, and in ROMs for embedded applications. A more general advantage of increased density is improved effectiveness of caches and instruction prefetch.

Computers with high code density often have complex instructions for procedure entry, parameterized returns, loops, etc. (therefore retroactively named *Complex Instruction Set Computers*, CISC). However, more typical, or frequent, "CISC" instructions merely combine a basic ALU operation, such as "add", with the access of one or more operands in memory (using addressing modes such as direct, indirect, indexed, etc.). Certain architectures may allow two or three operands (including the result) directly in memory or may be able to perform functions such as automatic pointer increment, etc. Software-implemented instruction sets may have even more complex and powerful instructions.

*Reduced instruction-set computers*, RISC, were first widely implemented during a period of rapidly growing memory subsystems. They sacrifice code density to simplify implementation circuitry, and try to increase performance via higher clock frequencies and more registers. A single RISC instruction typically performs only a single operation, such as an "add" of registers or a "load" from a memory location into a register. A RISC instruction set normally has a fixed instruction length, whereas a typical CISC instruction set has instructions of widely varying length. However, as RISC computers normally require more and often longer instructions to implement a given task, they inherently make less optimal use of bus bandwidth and cache memories.

Certain embedded RISC ISAs like Thumb and AVR32 typically exhibit very high density owing to a technique called code compression. This technique packs two 16-bit instructions into one 32-bit word, which is then unpacked at the decode stage and executed as two instructions.<sup>[16]</sup>

Minimal instruction set computers (MISC) are commonly a form of stack machine, where there are few separate instructions (8–32), so that multiple instructions can be fit into a single machine word. These types of cores often take little silicon to implement, so they can be easily realized in an FPGA (field-programmable gate array) or in a multi-core form. The code density of MISC is similar to the code density of RISC; the increased instruction density is offset by requiring more of the primitive instructions to do a task.<sup>[17]</sup>

There has been research into executable compression as a mechanism for improving code density. The mathematics of Kolmogorov complexity describes the challenges and limits of this.

In practice, code density is also dependent on the compiler. Most optimizing compilers have options that control whether to optimize code generation for execution speed or for code density. For instance GCC has the option `-Os` to optimize for small machine code size, and `-O3` to optimize for execution speed at the cost of larger machine code.

## Representation

The instructions constituting a program are rarely specified using their internal, numeric form (machine code); they may be specified by programmers using an assembly language or, more commonly, may be generated from high-level programming languages by compilers.<sup>[18]</sup>

## Design

---

The design of instruction sets is a complex issue. There were two stages in history for the microprocessor. The first was the CISC (complex instruction set computer), which had many different instructions. In the 1970s, however, places like IBM did research and found that many instructions in the set could be eliminated. The result was the RISC (reduced instruction set computer), an architecture that uses a smaller set of instructions. A simpler instruction set may offer the potential for higher speeds, reduced processor size, and reduced power consumption. However, a more complex set may optimize common operations, improve memory and cache efficiency, or simplify programming.

Some instruction set designers reserve one or more opcodes for some kind of system call or software interrupt. For example, MOS Technology 6502 uses  $00_H$ , Zilog Z80 uses the eight codes  $C7, CF, D7, DF, E7, EF, F7, FF_H$ <sup>[19]</sup> while Motorola 68000 use codes in the range  $4E40_H$ - $4E4F_H$ .<sup>[20]</sup>

Fast virtual machines are much easier to implement if an instruction set meets the Popek and Goldberg virtualization requirements.

The NOP slide used in immunity-aware programming is much easier to implement if the "unprogrammed" state of the memory is interpreted as a NOP.

On systems with multiple processors, non-blocking synchronization algorithms are much easier to implement if the instruction set includes support for something such as "fetch-and-add", "load-link/store-conditional" (LL/SC), or "atomic compare-and-swap".

## Instruction set implementation

---

A given instruction set can be implemented in a variety of ways. All ways of implementing a particular instruction set provide the same programming model, and all implementations of that instruction set are able to run the same executables. The various ways of implementing an instruction set give different tradeoffs between cost, performance, power consumption, size, etc.

When designing the microarchitecture of a processor, engineers use blocks of "hard-wired" electronic circuitry (often designed separately) such as adders, multiplexers, counters, registers, ALUs, etc. Some kind of register transfer language is then often used to describe the decoding and sequencing of each instruction of an ISA using this physical microarchitecture. There are two basic ways to build a control unit to implement this description (although many designs use middle ways or compromises):

1. Some computer designs "hardwire" the complete instruction set decoding and sequencing (just like the rest of the microarchitecture).
2. Other designs employ microcode routines or tables (or both) to do this, using ROMs or writable RAMs (writable control store), PLAs, or both.

Some microcoded CPU designs with a writable control store use it to allow the instruction set to be changed (for example, the Rekursiv processor and the Imsys Cjip).<sup>[21]</sup>

CPUs designed for reconfigurable computing may use field-programmable gate arrays (FPGAs).

An ISA can also be emulated in software by an interpreter. Naturally, due to the interpretation overhead, this is slower than directly running programs on the emulated hardware, unless the hardware running the emulator is an order of magnitude faster. Today, it is common practice for vendors of new ISAs or microarchitectures to make software emulators available to software developers before the hardware implementation is ready.

Often the details of the implementation have a strong influence on the particular instructions selected for the instruction set. For example, many implementations of the instruction pipeline only allow a single memory load or memory store per instruction, leading to a load–store architecture (RISC). For another example, some early ways of implementing the instruction pipeline led to a delay slot.

The demands of high-speed digital signal processing have pushed in the opposite direction—forcing instructions to be implemented in a particular way. For example, to perform digital filters fast enough, the MAC instruction in a typical digital signal processor (DSP) must use a kind of Harvard architecture that can fetch an instruction and two data words simultaneously, and it requires a single-cycle multiply–accumulate multiplier.

## See also

---

- Comparison of instruction set architectures
- Compressed instruction set
- Computer architecture
- Emulator
- Instruction set simulator
- Micro-operation
- No instruction set computing
- OVPsim – full systems simulator providing ability to create/model/emulate any instruction set using C and standard APIs
- Processor design
- Simulation
- Register transfer language (RTL)

## References

---

1. "GLOSSARY: Instruction Set Architecture (ISA)" (<https://web.archive.org/web/20231111175250/https://www.arm.com/glossary/isa>). *arm.com*. Archived from the original (<https://www.arm.com/glossary/isa>) on 2023-11-11. Retrieved 2024-02-03.
2. Pugh, Emerson W.; Johnson, Lyle R.; Palmer, John H. (1991). *IBM's 360 and Early 370 Systems* (<https://archive.org/details/ibms360early370s0000pugh>). MIT Press. ISBN 0-262-16123-0.

3. Chen, Crystal; Novick, Greg; Shimano, Kirk (December 16, 2006). "RISC Architecture: RISC vs. CISC" (<https://web.archive.org/web/20150221071744/http://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/>). *cs.stanford.edu*. Archived from the original (<http://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/>) on February 21, 2015. Retrieved February 21, 2015.
4. Schlansker, Michael S.; Rau, B. Ramakrishna (February 2000). "EPIC: Explicitly Parallel Instruction Computing". *Computer*. **33** (2): 37–45. doi:10.1109/2.820037 (<https://doi.org/10.1109/2.820037>).
5. Shaout, Adnan; Eldos, Taisir (Summer 2003). "On the Classification of Computer Architecture" (<https://www.researchgate.net/publication/267239549>). *International Journal of Science and Technology*. **14**: 3. Retrieved March 2, 2023.
6. Gilreath, William F.; Laplante, Phillip A. (December 6, 2012). *Computer Architecture: A Minimalist Perspective*. Springer Science+Business Media. ISBN 978-1-4615-0237-1.
7. Hennessy & Patterson 2003, p. 108.
8. Durand, Paul. "Instruction Set Architecture (ISA)" (<http://www.cs.kent.edu/~durand/CS0/Notes/Chapter05/isa.html>). *Introduction to Computer Science CS 0*.
9. "Renesas SH Instruction Set Summary" ([https://shared-ptr.com/sh\\_insns.html](https://shared-ptr.com/sh_insns.html)). *Shared-ptr.com*. Retrieved June 16, 2026.
10. Hennessy & Patterson 2003, p. 92.
11. Hennessy & Patterson 2003, p. 93.
12. Cocke, John; Markstein, Victoria (January 1990). "The evolution of RISC technology at IBM" (<https://acg.cis.upenn.edu/milom/cis501-Fall11/papers/cocke-RISC.pdf>) (PDF). *IBM Journal of Research and Development*. **34** (1): 4–11. doi:10.1147/rd.341.0004 (<https://doi.org/10.1147/rd.341.0004>). Retrieved 2026-02-06.
13. Page, Daniel (2009). "11. Compilers". *A Practical Introduction to Computer Architecture*. Springer. p. 464. Bibcode:2009pica.book.....P (<https://ui.adsabs.harvard.edu/abs/2009pica.book.....P>). ISBN 978-1-84882-255-9.
14. Venkat, Ashish; Tullsen, Dean M. (2014). *Harnessing ISA Diversity: Design of a Heterogeneous-ISA Chip Multiprocessor* (<http://dl.acm.org/citation.cfm?id=2665692>). 41st Annual International Symposium on Computer Architecture.
15. "Intel® 64 and IA-32 Architectures Software Developer's Manual" (<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>). Intel Corporation. Retrieved 5 October 2022.
16. Weaver, Vincent M.; McKee, Sally A. (2009). *Code density concerns for new architectures*. IEEE International Conference on Computer Design. CiteSeerX 10.1.1.398.1967 (<https://cite-seerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.398.1967>). doi:10.1109/ICCD.2009.5413117 (<https://doi.org/10.1109/ICCD.2009.5413117>).
17. "RISC vs. CISC" (<https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/>). *cs.stanford.edu*. Retrieved 2021-12-18.
18. Hennessy & Patterson 2003, p. 120.
19. Ganssle, Jack (February 26, 2001). "Proactive Debugging" (<https://www.embedded.com/electronics-blogs/break-points/4023293/Proactive-Debugging>). *embedded.com*.
20. *M68000 8-/16-/32-Bit Microprocessors User's Manual* (9 ed.). TRAP: Motorola. 1993. p. 4-188.
21. "Great Microprocessors of the Past and Present (V 13.4.0)" (<http://cpushack.net/CPU/cpu7.html>). *cpushack.net*. Retrieved 2014-07-25.

## Further reading

---

- Bowen, Jonathan P. (July–August 1985). "Standard Microprocessor Programming Cards". *Microprocessors and Microsystems*. **9** (6): 274–290. doi:10.1016/0141-9331(85)90116-4 (<https://doi.org/10.1016%2F0141-9331%2885%2990116-4>).
- Hennessy, John L.; Patterson, David A. (2003). *Computer Architecture: A Quantitative Approach* (<https://books.google.com/books?id=XX69oNsazH4C>) (Third ed.). Morgan Kaufmann Publishers. ISBN 1-55860-724-2. Retrieved 2023-03-04.

## External links

---

-  Media related to [Instruction set architectures](#) at Wikimedia Commons
- [Programming Textfiles: Bowen's Instruction Summary Cards](http://www.textfiles.com/programming/CARDS/) (<http://www.textfiles.com/programming/CARDS/>)
- [Mark Smotherman's Historical Computer Designs Page](https://people.computing.clemson.edu/~mark/hist.html) (<https://people.computing.clemson.edu/~mark/hist.html>)

---

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Instruction\\_set\\_architecture&oldid=1360461049](https://en.wikipedia.org/w/index.php?title=Instruction_set_architecture&oldid=1360461049)"